

Trabajo Colaborativo en Parejas

1. INVESTIGACIÓN TEÓRICA

1. Fork

Definición:

Copia completa de un repositorio bajo el control de otro usuario, permitiendo desarrollo independiente sin permisos de escritura en el original.

Utilidad:

Permite contribuir a proyectos de código abierto sin permisos directos, experimentar con el código y proponer cambios mediante Pull Requests.

Ejemplo Práctico:

Un desarrollador hace fork del proyecto 'linux' en GitHub, realiza mejoras en su copia, y luego crea un PR para que sea integrado en el proyecto original.

Fuente: GitHub - Fork a Repository

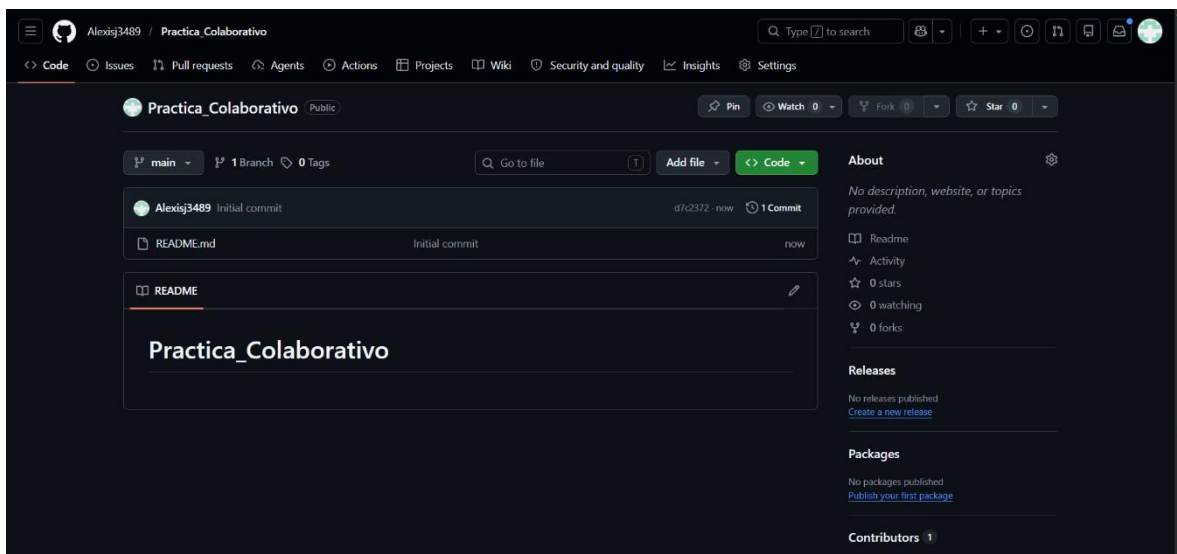
2. DEMOSTRACIÓN PRÁCTICA

En esta sección se documentan mediante explicaciones los pasos realizados durante la práctica colaborativa. Cada sección describe la actividad ejecutada y el resultado obtenido.

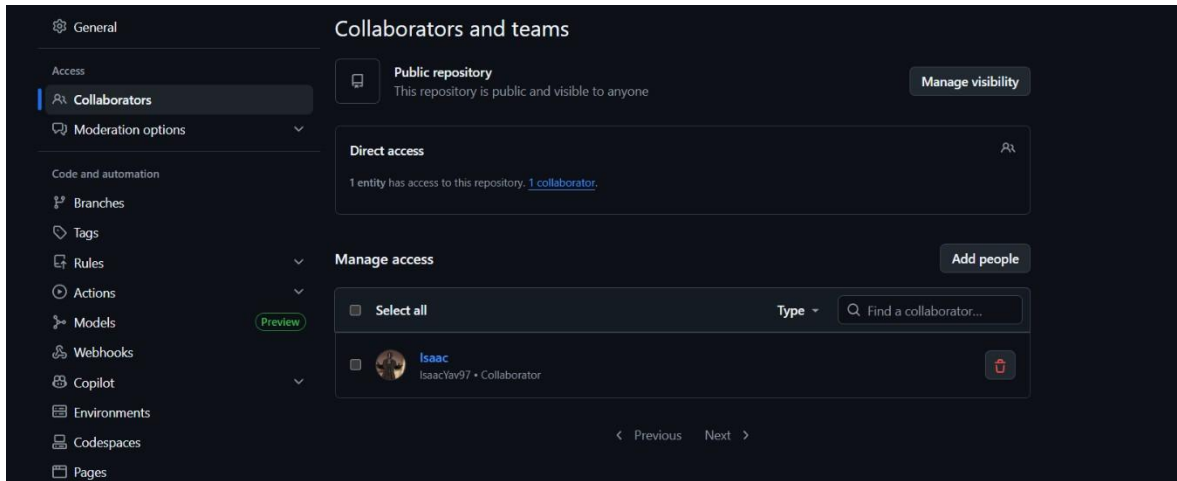
1. Creación del Repositorio en GitHub

Actividad: El Estudiante A crea un nuevo repositorio público en GitHub con un archivo README.md inicial que describe el proyecto e invita al colaborador y el acepta.

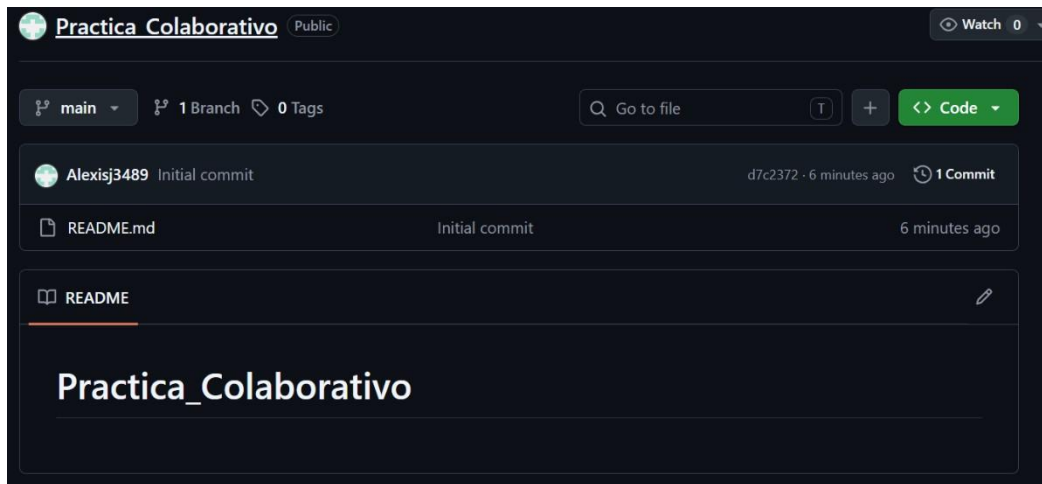
Estudiante A



Estudiante B



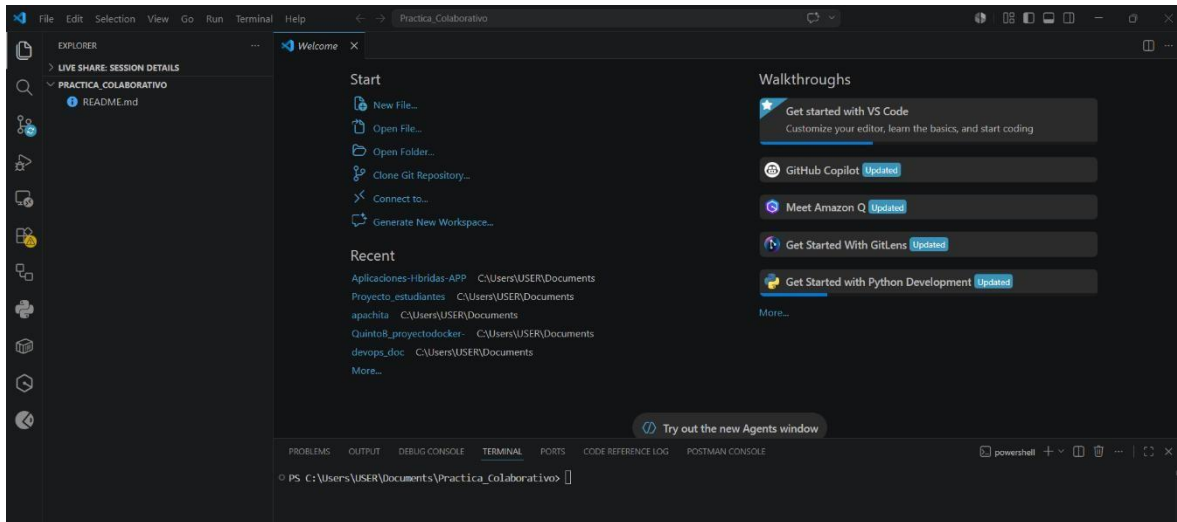
Resultado Obtenido: Repositorio creado exitosamente en GitHub con la URL: https://github.com/Alexisj3489/Practica_Colaborativo.git



2. Clonación y Configuración Inicial

Actividad: Ambos estudiantes clonan el repositorio en sus máquinas locales usando 'git clone'. Se configura git con nombres de usuarios.

Estudiante A



Estudiante B

```
PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git> git clone https://github.com/Alexisj3489/Practica_Colaborativo.git
Cloning into 'Practica_Colaborativo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Resultado Obtenido: El repositorio está disponible localmente en ambas máquinas con todo el historial y configuración lista para trabajar.

3. Creación de Ramas de Trabajo

Actividad: el Estudiante A creó la rama feature-estudianteA y trabajó en una parte del sistema, mientras que el Estudiante B creó la rama feature-estudianteB para desarrollar otra funcionalidad.

Estudiante A

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  CODE REFERENCE LOG  POSTMAN CONSOLE

PS C:\Users\USER\Documents\Practica_Colaborativo> python --version
Python 3.14.5
PS C:\Users\USER\Documents\Practica_Colaborativo> git checkout -b feature-estudianteA
Switched to a new branch 'feature-estudianteA'
PS C:\Users\USER\Documents\Practica_Colaborativo> git branch
* feature-estudianteA
main
PS C:\Users\USER\Documents\Practica_Colaborativo>
```

Estudiante B

```
(venv) PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git\Practica_Colaborati
• vo> git checkout -b feature-estudianteB
Switched to a new branch 'feature-estudianteB'
(venv) PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git\Practica_Colaborati
• vo> git remote -v
origin https://github.com/Alexisj3489/Practica_Colaborativo.git (fetch)
origin https://github.com/Alexisj3489/Practica_Colaborativo.git (push)
(venv) PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git\Practica_Colaborati
• vo> git branch
* feature-estudianteB
main
```

Resultado Obtenido: Se crean dos ramas independientes en el repositorio remoto, cada una lista para desarrollo paralelo.

4. Commits por Ambos Integrantes

Actividad: Estudiante B realiza commits en su rama con mensajes descriptivos como 'Implementar login con email'. Estudiante A (intercambiando roles) realiza commits en su rama.

Estudiante A

```
main
• PS C:\Users\USER\Documents\Practica_Colaborativo> git add .
• PS C:\Users\USER\Documents\Practica_Colaborativo> git commit -m "primer push de la rama A"
[feature-estudianteA 157ba08] primer push de la rama A
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 .github/workflows/ci.yml
```

Estudiante B

```
(venv) PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git\Practica_Colaborati
vo> git add .
(venv) PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git\Practica_Colaborati
vo> git commit -m "Se agrega del estudiante B"
[feature-estudianteB 81e9db3] Se agrega del estudiante B
1 file changed, 16 insertions(+)
create mode 100644 app/estudiantes.py
```

Resultado Obtenido: El repositorio contiene múltiples commits de ambos estudiantes documentando el desarrollo de diferentes características.

5. Publicación de Cambios al Repositorio Remoto

Actividad: Ambos estudiantes usan 'git push' para enviar sus ramas locales al repositorio remoto en GitHub, haciendo visible su trabajo a su compañero.

Estudiante A

```
• PS C:\Users\USER\Documents\Practica_Colaborativo> git push origin feature-estudianteA
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 5 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (5/5), 381 bytes | 190.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-estudianteA' on GitHub by visiting:
remote:   https://github.com/Alexisj3489/Practica_Colaborativo/pull/new/feature-estudianteA
remote:
To https://github.com/Alexisj3489/Practica_Colaborativo.git
* [new branch]   feature-estudianteA -> feature-estudianteA
• PS C:\Users\USER\Documents\Practica_Colaborativo>
```

Estudiante B

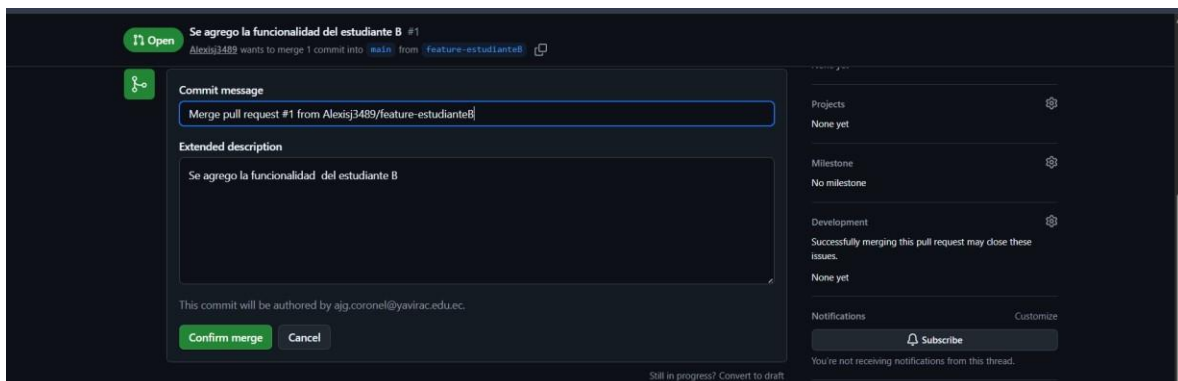
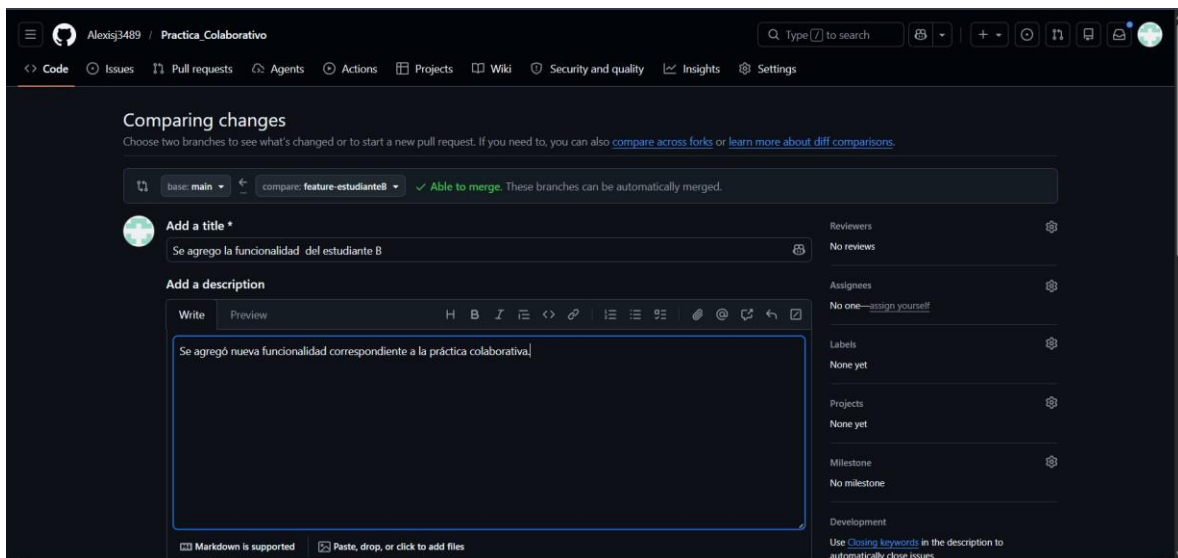
```
(venv) PS C:\Users\usuario\Desktop\YO\INSTITUTO\YAVIRAC\SEMESTRE 5\BYRON MORENO\practica_git\Practica_Colaborati
vo> git push origin feature-estudianteB
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 538 bytes | 67.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-estudianteB' on GitHub by visiting:
remote:   https://github.com/Alexisj3489/Practica_Colaborativo/pull/new/feature-estudianteB
remote:
To https://github.com/Alexisj3489/Practica_Colaborativo.git
* [new branch]      feature-estudianteB -> feature-estudianteB
```

Resultado Obtenido: Ambas ramas están disponibles en GitHub con todos los commits sincronizados desde las máquinas locales.

6. Creación de Pull Requests

Actividad: Cada estudiante crea un Pull Request en GitHub con una descripción clara de los cambios realizados, esperando revisión antes de integrar a main.

Estudiante A



Conversation 1 | Commits 1 | Checks 0 | Files changed 1

All commits

Filter files...

app

estudiantes.py

app/estudiantes.py

```
@@ -0,0 +1,16 @@
1 + def registrar_estudiante(nombre, edad):
2 +     if nombre == "" or edad <= 0:
3 +         return "Datos inválidos"
4 +     return f"Estudiante {nombre} registrado con {edad} años"
5 +
6 +
7 + def calcular_promedio(notas):
8 +     if len(notas) == 0:
9 +         return 0
10 +     return sum(notas) / len(notas)
11 +
12 +
13 + def aprobo(promedio):
14 +     if promedio >= 7:
15 +         return "Aprobado"
16 +     return "Reprobado"
```

Alexisj3489 commented now

View reviewed changes

Alexisj3489 left a comment

Owner Author ...

su codigo esta correctamente sin ningun error

Estudiante B

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main | compare: feature-estudianteA | ✓ Able to merge. These branches can be automatically merged.

Add a title *

Se agrego la funcionalidad del estudiante A

Add a description

Write | Preview

Se agregó nueva funcionalidad correspondiente a la práctica colaborativa

Markdown is supported | Paste, drop, or click to add files

Reviewers: No reviews

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Use [Closing keywords](#) in the description to automatically close issues

Create pull request

Merged IsaacYav97 merged 1 commit into main from feature-estudianteA 2 minutes ago

Conversation 1 | Commits 1 | Checks 0 | Files changed 1

All commits

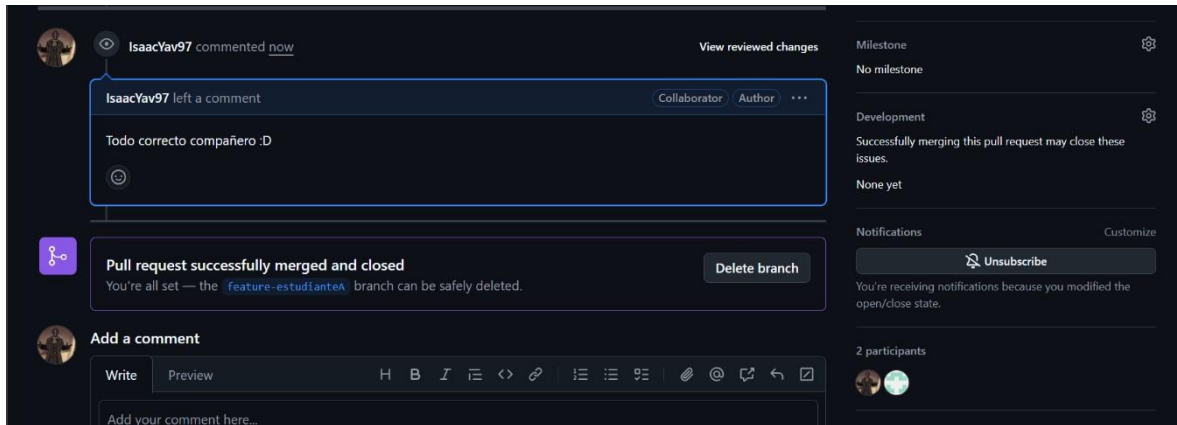
Filter files...

.github/workflows

ci.yml

.github/workflows/ci.yml

Whitespace-only changes.

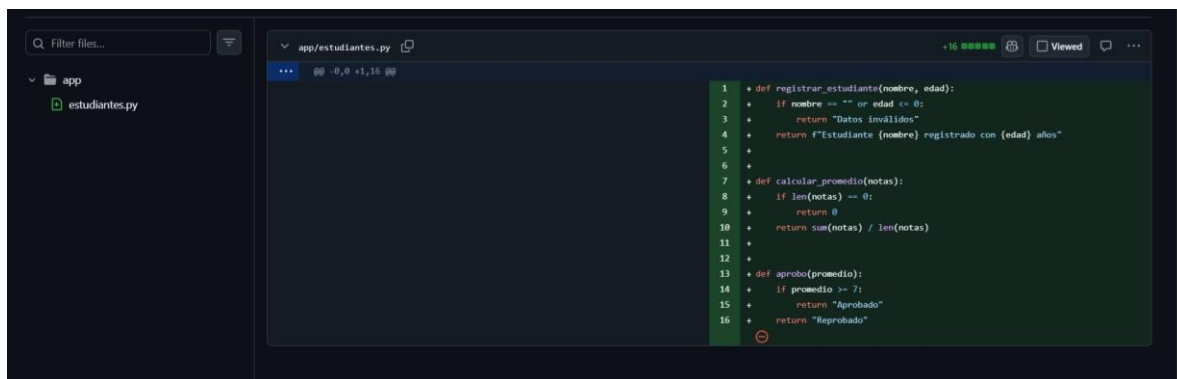


Resultado Obtenido: Se crean dos PRs en GitHub esperando aprobación, incluyendo cambios de ambos estudiantes.

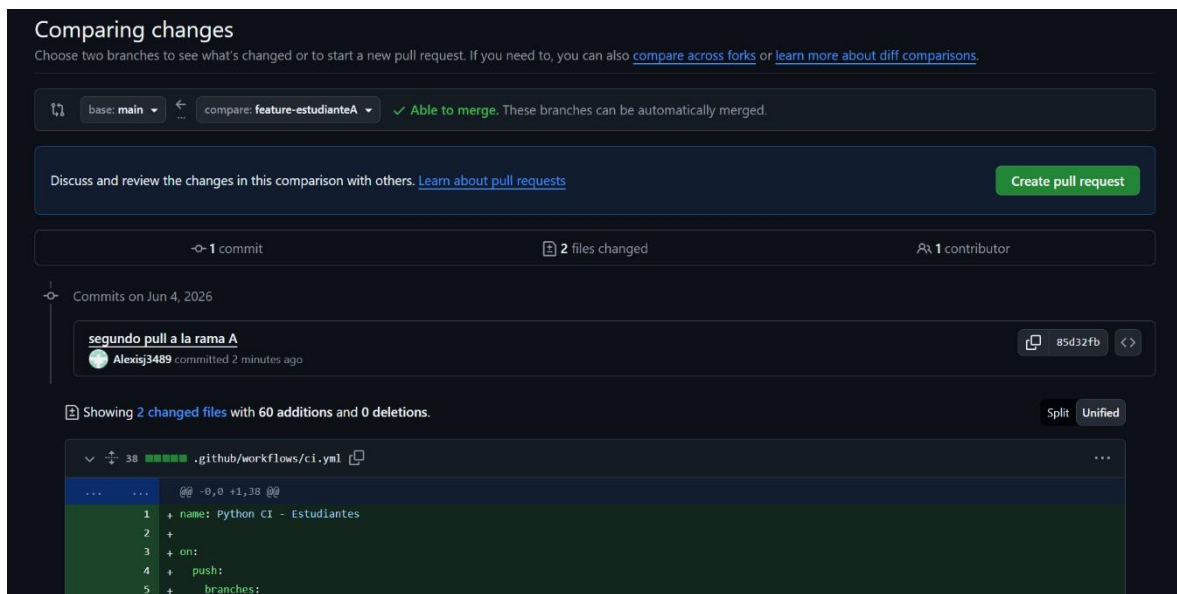
7. Revisión de Código

Actividad: Ambos estudiantes revisan el código del otro mediante los comentarios en el PR, sugieren mejoras y verifican que el código sea de buena calidad.

Estudiante A



Estudiante B



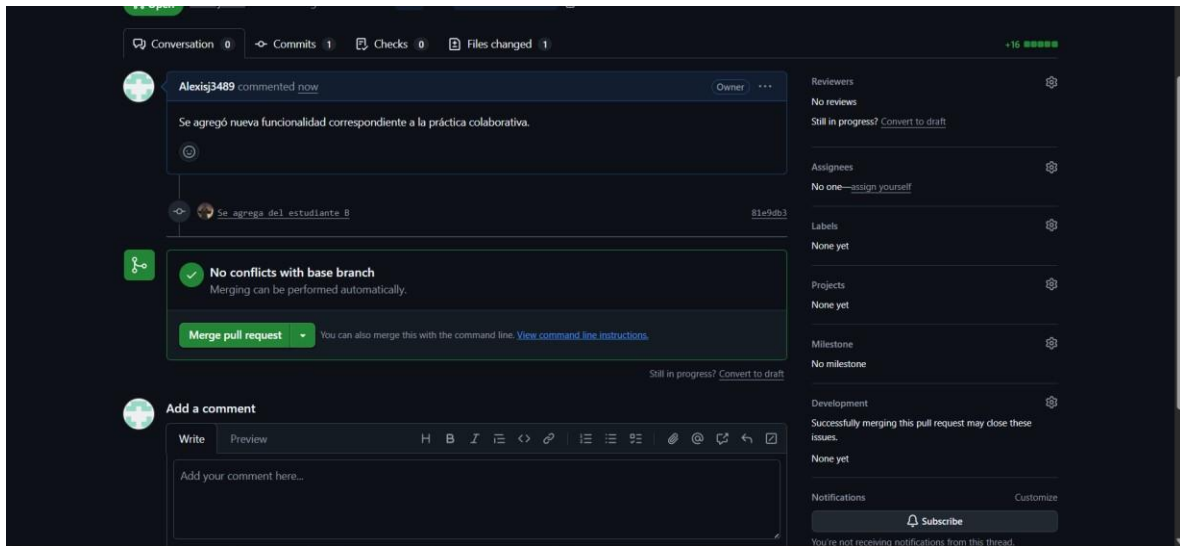
Resultado Obtenido: Cada PR incluye comentarios constructivos y validación del código antes de integración.

8. Aprobación de Pull Requests

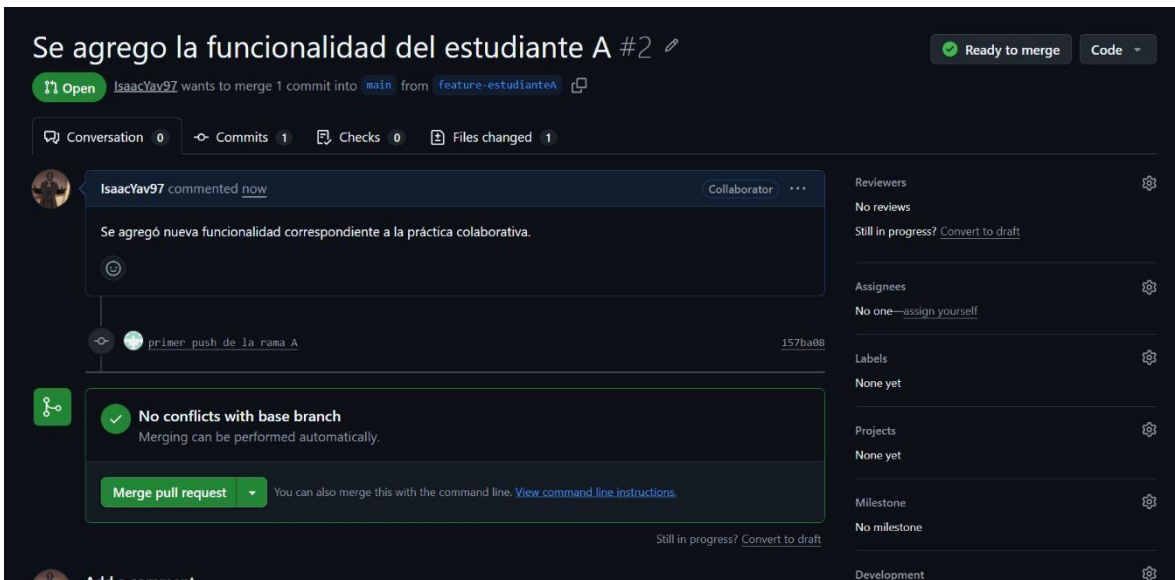
Actividad: Después de la revisión satisfactoria, ambos estudiantes aprueban los PRs del otro dando permiso para proceder con el merge.

Resultado Obtenido: Los PRs están aprobados y listos para ser mergeados a la rama main.

Estudiante A



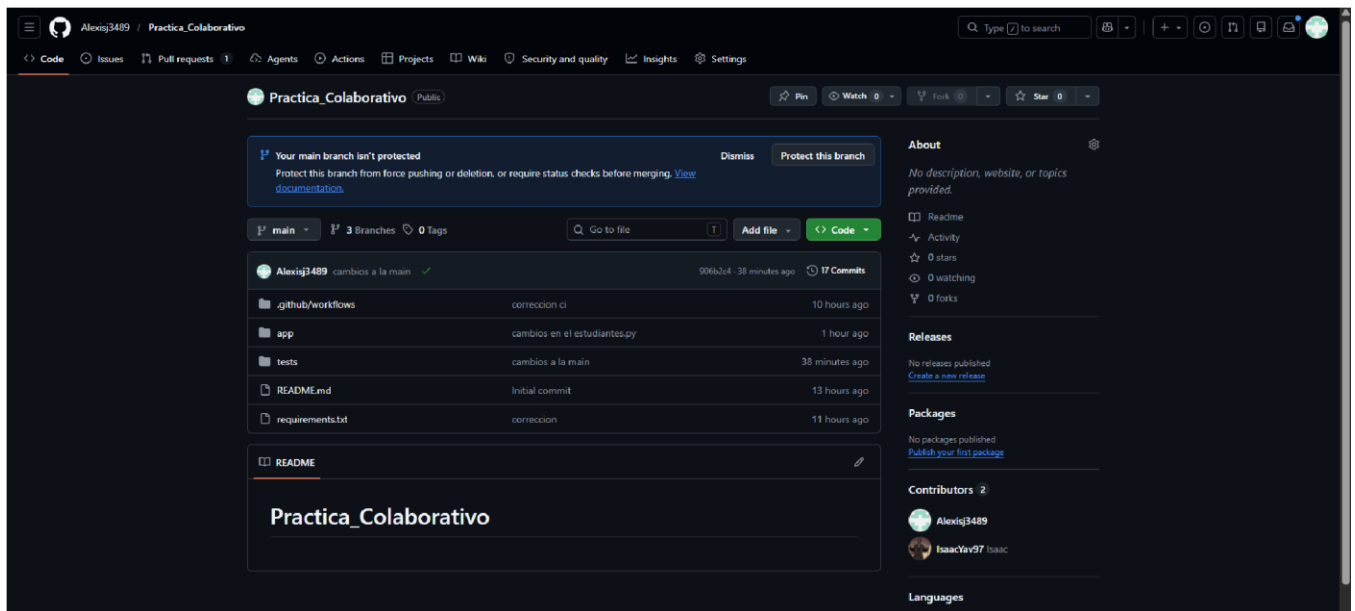
Estudiante B



9. Ejecución de Merge

Actividad: Se ejecutan los merges de ambas ramas a main. Se utiliza 'Squash and merge' para mantener el historial limpio.

Estudiante A Y B

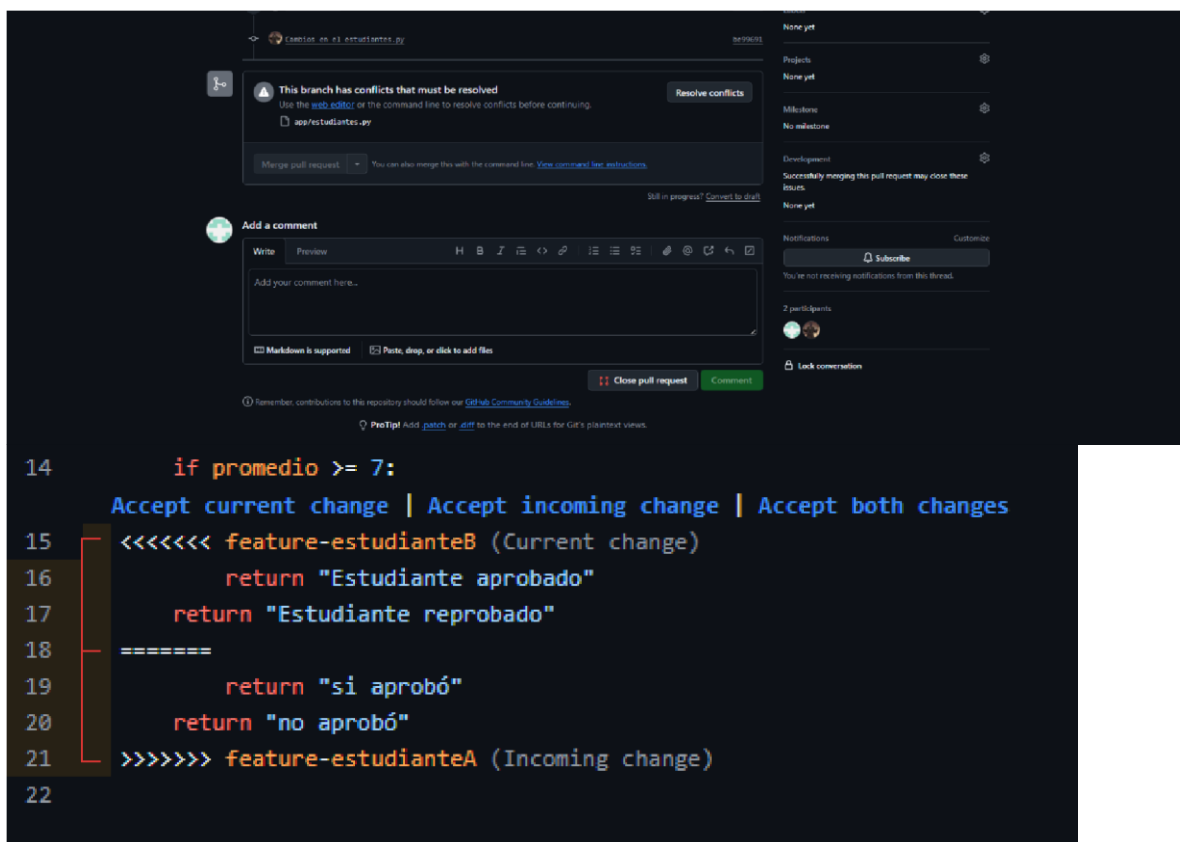


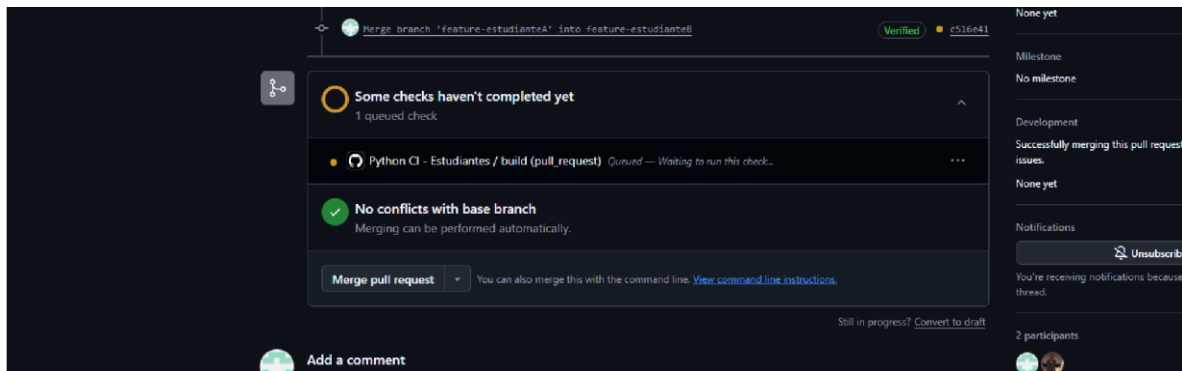
Resultado Obtenido: Ambas características están integradas en main, y el código está unificado.

10. Generación de Conflictos de Integración

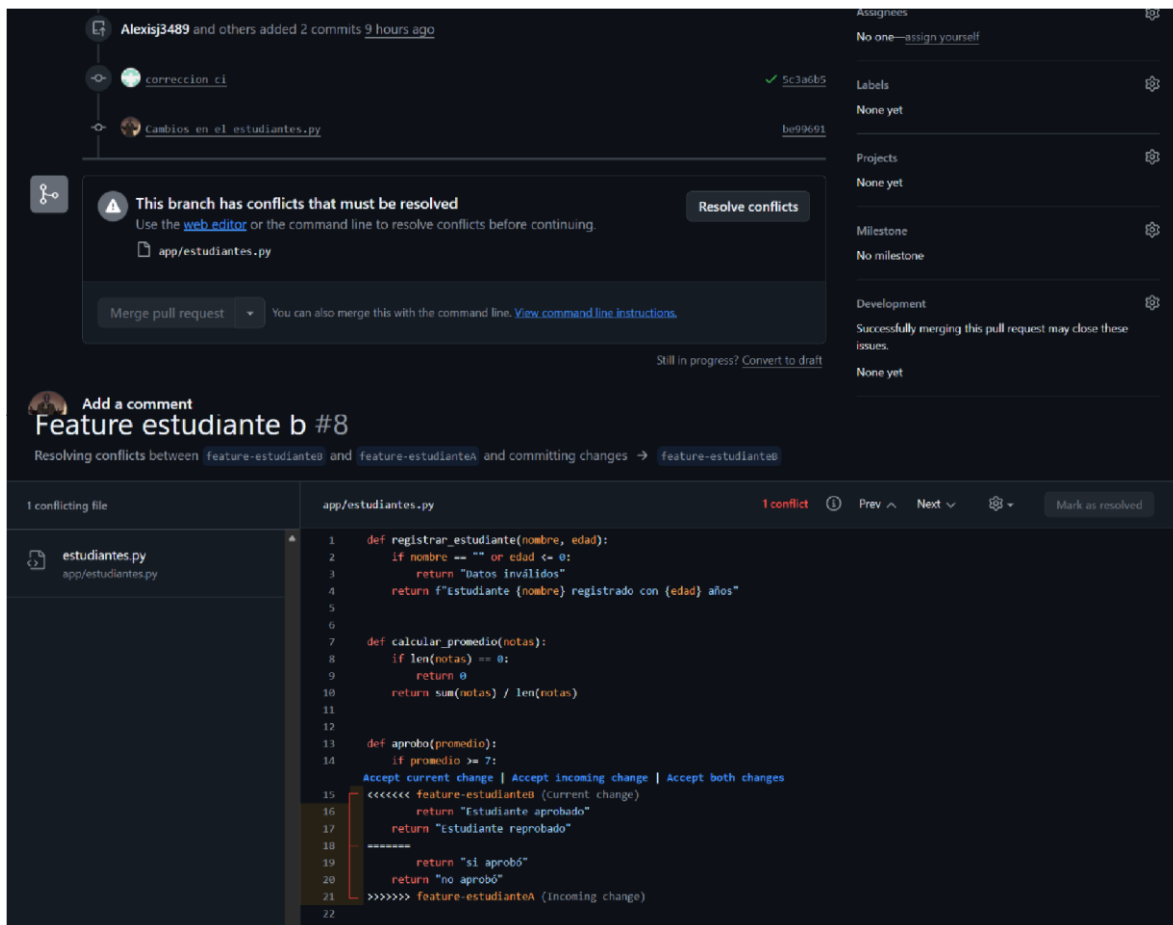
Actividad: De manera controlada, ambos estudiantes modifican las mismas líneas en un archivo, generando un conflicto intencional al intentar hacer merge.

Estudiante A





Estudiante B



Resultado Obtenido: Git detecta el conflicto y requiere resolución manual de los cambios conflictivos.

11. Resolución de Conflictos

Actividad: Ambos estudiantes practican resolver conflictos editando manualmente el archivo, decidiendo qué código mantener y eliminando los marcadores de conflicto.

Estudiante A

```
1 def registrar_estudiante(nombre, edad):
2     if nombre == "" or edad <= 0:
3         return "Datos inválidos"
4     return f"Estudiante {nombre} registrado con {edad} años"
5
6
7 def calcular_promedio(notas):
8     if len(notas) == 0:
9         return 0
10    return sum(notas) / len(notas)
11
12
13 def aprobo(promedio):
14     if promedio >= 7:
15         return "Estudiante aprobado"
16     return "Estudiante reprobado"
17
```

Estudiante B

Feature estudiante b #8

Resolving conflicts between feature-estudianteB and feature-estudianteA and committing changes → feature-estudianteB

Commit merge

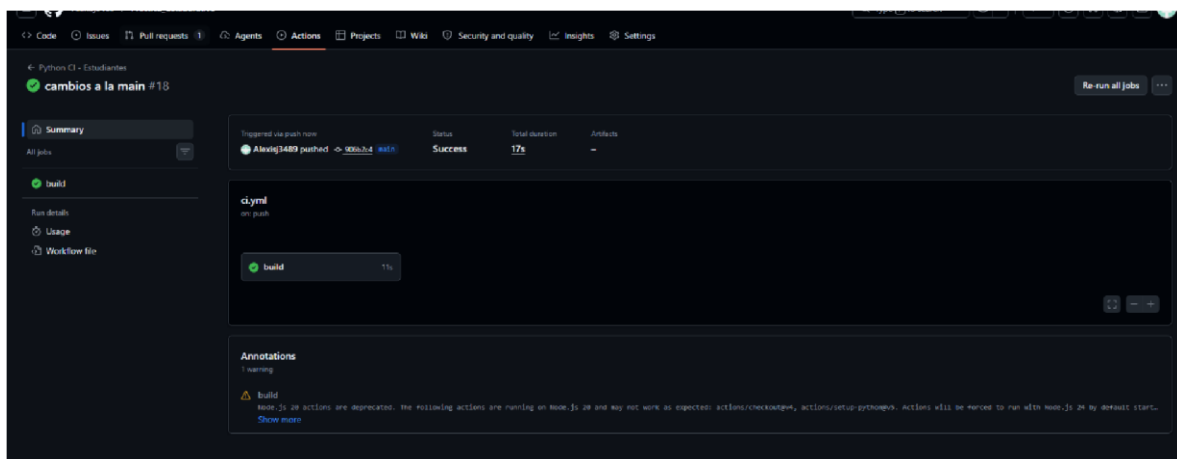
```
1 def registrar_estudiante(nombre, edad):
2     if nombre == "" or edad <= 0:
3         return "Datos inválidos"
4     return f"Estudiante {nombre} registrado con {edad} años"
5
6
7 def calcular_promedio(notas):
8     if len(notas) == 0:
9         return 0
10    return sum(notas) / len(notas)
11
12
13 def aprobo(promedio):
14     if promedio >= 7:
15         return "Estudiante aprobado"
16     return "Estudiante reprobado"
17
```

Resultado Obtenido: El conflicto se resuelve exitosamente y el merge se completa con el código combinado correctamente.

12. Configuración y Ejecución de GitHub Actions

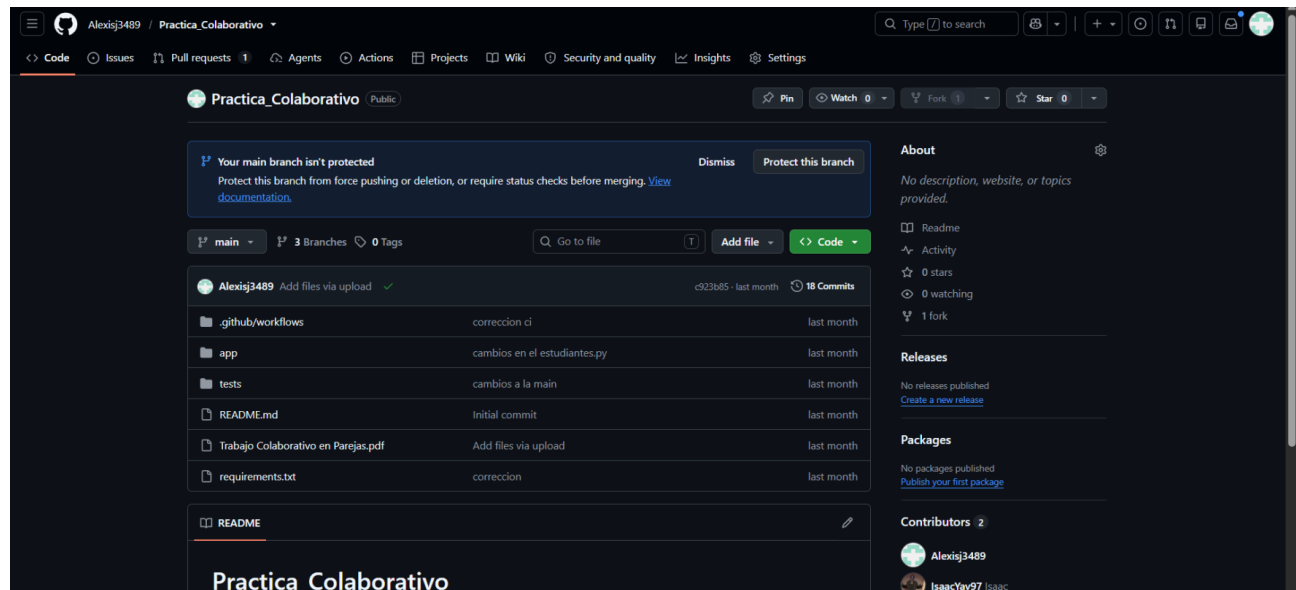
Actividad: Se configura un workflow de GitHub Actions que ejecuta automáticamente tests y validación de código en cada push y PR.

Estudiante A Y B



Resultado Obtenido: El workflow se ejecuta exitosamente en cada cambio, validando la calidad del código antes de permitir integración.

15. Prueba de que se realizo el Fork del visitante



3. CONCLUSIONES

Conclusión del Estudiante A:

Durante esta práctica de Git colaborativo, obtuve una comprensión profunda del flujo de trabajo en equipo moderno. Aprendí que la revisión de código no es solo una validación técnica, sino una oportunidad para aprender de mis compañeros y mejorar la calidad general del proyecto. El proceso de resolver conflictos de integración me enseñó la importancia de comunicación clara y decisiones bien fundamentadas cuando hay cambios conflictivos. Además, implementar GitHub Actions me mostró cómo la automatización puede mejorar significativamente la confiabilidad y eficiencia del desarrollo. Este conocimiento será invaluable en mi carrera profesional como desarrollador.

Conclusión del Estudiante B:

La práctica colaborativa de Git ha transformado mi perspectiva sobre el desarrollo de software. Comprendí que trabajar en equipo requiere disciplina en nombrar ramas, escribir commits descriptivos y mantener una comunicación constante. La experiencia de crear y gestionar Pull Requests me hizo apreciar la importancia de documentar cambios claramente para facilitar la revisión. Al resolver conflictos de integración, aprendí a tomar decisiones técnicas bien consideradas. Esta práctica me preparó adecuadamente para trabajar en proyectos reales donde la colaboración efectiva es esencial para el éxito del equipo.

4. BIBLIOGRAFÍA

1. Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.
2. GitHub Docs. (2024). GitHub - Official Documentation. <https://docs.github.com/>

3. Fowler, M. (2014). Continuous Integration. Retrieved from <https://martinfowler.com/articles/continuousIntegration.html>
4. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
5. Git SCM. (2024). Git - Official Documentation. <https://git-scm.com/doc>
6. GitHub Actions Documentation. (2024). Automating your workflow with GitHub Actions. <https://docs.github.com/en/actions>
7. Atlassian. (2024). Git Tutorials and Training. <https://www.atlassian.com/git/tutorials>
8. Microsoft Learn. (2024). Introduction to Git. <https://learn.microsoft.com/en-us/training/modules/intro-to-git/>
9. Free Code Camp. (2024). Git and GitHub Tutorial for Beginners. YouTube.
10. Linux Foundation. (2024). Introduction to Linux - LF101x. <https://www.edx.org/course/introduction-to-linux>

INFORMACIÓN DE ENTREGA

INTEGRANTE A	Alexis Josue Coronel Guanotasig
INTEGRANTE B	Rafael Isaac Pacheco Quevedo
CARRERA	Desarrollo de software
PARALELO	B
FECHA	05/06/2026
URL REPOSITORIO GITHUB	https://github.com/Alexisj3489/Practica_Colaborativo.git